In [1]:
```
import pandas as pd
import numpy as np
import requests
import json
import time
import warnings
warnings.filterwarnings('ignore')

# Your API endpoint
API_URL = "http://localhost:8000/analyze"

# The exact 20 Golden Features your model uses
GOLDEN_FEATURES = [
    'Packet Length Variance',
    'Bwd Packet Length Max',
    'Max Packet Length',
    'Total Length of Fwd Packets',
    'Packet Length Mean',
    'Fwd Packet Length Mean',
    'Fwd IAT Std',
    'Fwd Packet Length Max',
    'Bwd Header Length',
    'Fwd Header Length',
    'PSH Flag Count',
    'Flow IAT Std',
    'Init_Win_bytes_backward',
    'Flow IAT Mean',
    'Bwd Packet Length Min',
    'Flow IAT Max',
    'min_seg_size_forward',
    'Min Packet Length',
    'Init_Win_bytes_forward',
    'act_data_pkt_fwd'
]

print("Setup complete")
```

```
Setup complete
```

In [2]:
```
 # UPDATE THIS PATH to your actual CSV file
CSV_PATH = "/Users/kruthikhavishali/Downloads/all_live_traffic.csv"

# Load it
df = pd.read_csv(CSV_PATH, encoding='utf-8', encoding_errors='repla
df.columns = df.columns.str.strip()

print(f"Total flows captured: {len(df)}")
print(f"Columns available: {df.shape[1]}")

# Check which of our 20 features are present
missing = [f for f in GOLDEN_FEATURES if f not in df.columns]
present = [f for f in GOLDEN_FEATURES if f in df.columns]

print(f"\nGolden features found: {len(present)}/20")
if missing:
```

```
    print(f"Missing features: {missing}")
else:
    print("All 20 Golden Features present!")
```

```
Total flows captured: 413
Columns available: 347

Golden features found: 0/20
Missing features: ['Packet Length Variance', 'Bwd Packet Length Ma
x', 'Max Packet Length', 'Total Length of Fwd Packets', 'Packet Leng
th Mean', 'Fwd Packet Length Mean', 'Fwd IAT Std', 'Fwd Packet Lengt
h Max', 'Bwd Header Length', 'Fwd Header Length', 'PSH Flag Count',
'Flow IAT Std', 'Init_Win_bytes_backward', 'Flow IAT Mean', 'Bwd Pac
ket Length Min', 'Flow IAT Max', 'min_seg_size_forward', 'Min Packet
Length', 'Init_Win_bytes_forward', 'act_data_pkt_fwd']
```

In [3]:

```
# CICFlowMeter sometimes uses slightly different column names
# This maps them to what your model expects
COLUMN_MAP = {
    'Fwd Header Length.1': 'Fwd Header Length',  # duplicate fix
    'Init_Win_bytes_backward': 'Init_Win_bytes_backward',
    'Init_Win_bytes_forward': 'Init_Win_bytes_forward',
}

# Apply mapping
df = df.rename(columns=COLUMN_MAP)

# Extract only the 20 Golden Features
# Handle any that are still missing by filling with 0
for feat in GOLDEN_FEATURES:
    if feat not in df.columns:
        print(f"WARNING: '{feat}' not found — filling with 0")
        df[feat] = 0

# Select only golden features
df_golden = df[GOLDEN_FEATURES].copy()

# Clean: replace inf and nan with 0
df_golden = df_golden.replace([np.inf, -np.inf], np.nan).fillna(0)

print(f"Ready to analyze {len(df_golden)} flows")
print(f"\nSample flow (first row):")
print(df_golden.iloc[0])
```

```
WARNING: 'Packet Length Variance' not found — filling with 0
WARNING: 'Bwd Packet Length Max' not found — filling with 0
WARNING: 'Max Packet Length' not found — filling with 0
WARNING: 'Total Length of Fwd Packets' not found — filling with 0
WARNING: 'Packet Length Mean' not found — filling with 0
WARNING: 'Fwd Packet Length Mean' not found — filling with 0
WARNING: 'Fwd IAT Std' not found — filling with 0
WARNING: 'Fwd Packet Length Max' not found — filling with 0
WARNING: 'Bwd Header Length' not found — filling with 0
WARNING: 'Fwd Header Length' not found — filling with 0
WARNING: 'PSH Flag Count' not found — filling with 0
WARNING: 'Flow IAT Std' not found — filling with 0
WARNING: 'Init_Win_bytes_backward' not found — filling with 0
WARNING: 'Flow IAT Mean' not found — filling with 0
WARNING: 'Bwd Packet Length Min' not found — filling with 0
WARNING: 'Flow IAT Max' not found — filling with 0
WARNING: 'min_seg_size_forward' not found — filling with 0
WARNING: 'Min Packet Length' not found — filling with 0
WARNING: 'Init_Win_bytes_forward' not found — filling with 0
WARNING: 'act_data_pkt_fwd' not found — filling with 0
Ready to analyze 413 flows

Sample flow (first row):
Packet Length Variance         0
Bwd Packet Length Max          0
Max Packet Length              0
Total Length of Fwd Packets    0
Packet Length Mean             0
Fwd Packet Length Mean         0
Fwd IAT Std                    0
Fwd Packet Length Max          0
Bwd Header Length              0
Fwd Header Length              0
PSH Flag Count                 0
Flow IAT Std                   0
Init_Win_bytes_backward        0
Flow IAT Mean                  0
Bwd Packet Length Min          0
Flow IAT Max                   0
min_seg_size_forward           0
Min Packet Length              0
Init_Win_bytes_forward         0
act_data_pkt_fwd               0
Name: 0, dtype: int64
```

In [4]:

```python
def analyze_flow(flow_dict):
    """Send one flow to FastAPI and get prediction."""
    payload = {"features": flow_dict}
    try:
        response = requests.post(API_URL, json=payload, timeout=30)
        if response.status_code == 200:
            return response.json()
        else:
            return {"error": f"HTTP {response.status_code}", "detai
    except requests.exceptions.ConnectionError:
        return {"error": "API not running — start backend first"}
```

```
    except Exception as e:
        return {"error": str(e)}

# Test with first flow
print("Testing API connection with Flow #0...")
test_flow = df_golden.iloc[0].to_dict()
result = analyze_flow(test_flow)

if "error" in result:
    print(f"ERROR: {result['error']}")
    print("Make sure your backend is running: uvicorn app.main:app
else:
    print(f"API connected successfully!")
    print(f"Flow #0 → {result['label']} ({result['probability']*100
    print(f"Inference time: {result['inference_ms']}ms")
```

```
Testing API connection with Flow #0...
ERROR: HTTPConnectionPool(host='localhost', port=8000): Read timed o
ut. (read timeout=30)
Make sure your backend is running: uvicorn app.main:app --reload
```

In [5]:

```
# Analyze all flows and collect results
print(f"Analyzing {len(df_golden)} real captured flows...\n")

results = []
attack_count = 0
benign_count = 0
error_count = 0

for i, (idx, row) in enumerate(df_golden.iterrows()):
    flow_dict = row.to_dict()
    result = analyze_flow(flow_dict)

    if "error" in result:
        error_count += 1
        continue

    results.append({
        'flow_index': i,
        'label': result['label'],
        'prediction': result['prediction'],
        'probability': result['probability'],
        'inference_ms': result['inference_ms'],
        'top_shap_feature': result['feature_names'][
            result['shap_values'].index(max(result['shap_values']))
        ]
    })

    if result['prediction'] == 1:
        attack_count += 1
        print(f"  Flow #{i:3d} → ⚠️  ATTACK  ({result['probability'
    else:
        benign_count += 1
        if i % 10 == 0:  # only print every 10th benign to avoid sp
            print(f"  Flow #{i:3d} → ✓  BENIGN  ({result['probabili
```

```
    time.sleep(0.1)  # small delay to not overwhelm API

print(f"\n{'='*50}")
print(f"ANALYSIS COMPLETE")
print(f"{'='*50}")
print(f"Total flows analyzed : {len(results)}")
print(f"ATTACK flows         : {attack_count}")
print(f"BENIGN flows         : {benign_count}")
print(f"Errors               : {error_count}")
print(f"Attack rate          : {attack_count/len(results)*100:.2f}%
```

```
Analyzing 413 real captured flows...

  Flow #  0 → ✓  BENIGN  (0.402%)
  Flow # 10 → ✓  BENIGN  (0.402%)
  Flow # 20 → ✓  BENIGN  (0.402%)
  Flow # 30 → ✓  BENIGN  (0.402%)
  Flow # 40 → ✓  BENIGN  (0.402%)
  Flow # 50 → ✓  BENIGN  (0.402%)
  Flow # 60 → ✓  BENIGN  (0.402%)
  Flow # 70 → ✓  BENIGN  (0.402%)
  Flow # 80 → ✓  BENIGN  (0.402%)
  Flow # 90 → ✓  BENIGN  (0.402%)
  Flow #100 → ✓  BENIGN  (0.402%)
  Flow #110 → ✓  BENIGN  (0.402%)
  Flow #120 → ✓  BENIGN  (0.402%)
  Flow #130 → ✓  BENIGN  (0.402%)
  Flow #140 → ✓  BENIGN  (0.402%)
  Flow #150 → ✓  BENIGN  (0.402%)
  Flow #160 → ✓  BENIGN  (0.402%)
  Flow #170 → ✓  BENIGN  (0.402%)
  Flow #180 → ✓  BENIGN  (0.402%)
  Flow #190 → ✓  BENIGN  (0.402%)
  Flow #200 → ✓  BENIGN  (0.402%)
  Flow #210 → ✓  BENIGN  (0.402%)
  Flow #220 → ✓  BENIGN  (0.402%)
  Flow #230 → ✓  BENIGN  (0.402%)
  Flow #240 → ✓  BENIGN  (0.402%)
  Flow #250 → ✓  BENIGN  (0.402%)
  Flow #260 → ✓  BENIGN  (0.402%)
  Flow #270 → ✓  BENIGN  (0.402%)
  Flow #280 → ✓  BENIGN  (0.402%)
  Flow #290 → ✓  BENIGN  (0.402%)
  Flow #300 → ✓  BENIGN  (0.402%)
  Flow #310 → ✓  BENIGN  (0.402%)
  Flow #320 → ✓  BENIGN  (0.402%)
  Flow #330 → ✓  BENIGN  (0.402%)
  Flow #340 → ✓  BENIGN  (0.402%)
  Flow #350 → ✓  BENIGN  (0.402%)
  Flow #360 → ✓  BENIGN  (0.402%)
  Flow #370 → ✓  BENIGN  (0.402%)
  Flow #380 → ✓  BENIGN  (0.402%)
  Flow #390 → ✓  BENIGN  (0.402%)
  Flow #400 → ✓  BENIGN  (0.402%)
  Flow #410 → ✓  BENIGN  (0.402%)

================================================
ANALYSIS COMPLETE
================================================
Total flows analyzed : 413
ATTACK flows         : 0
BENIGN flows         : 413
Errors               : 0
Attack rate          : 0.00%
```

In [6]:
```
# Create results dataframe
results_df = pd.DataFrame(results)
```

```
print("\n=== LIVE NETWORK ANALYSIS REPORT ===\n")
print(f"Network interface: en0 (your WiFi)")
print(f"Flows captured: {len(results_df)}")
print(f"Attacks detected: {attack_count} ({attack_count/len(results
print(f"Benign flows: {benign_count} ({benign_count/len(results_df)

if attack_count > 0:
    print(f"\n⚠️  ATTACKS DETECTED:")
    attacks = results_df[results_df['prediction'] == 1]
    for _, row in attacks.iterrows():
        print(f"  Flow #{row['flow_index']} — {row['probability']*1
else:
    print(f"\n✓ All flows classified as BENIGN — your network looks

print(f"\nAverage inference time: {results_df['inference_ms'].mean(
print(f"Max inference time: {results_df['inference_ms'].max():.3f}m

# Save results
results_df.to_csv('live_analysis_results.csv', index=False)
print(f"\nResults saved to: live_analysis_results.csv")
```

```
=== LIVE NETWORK ANALYSIS REPORT ===

Network interface: en0 (your WiFi)
Flows captured: 413
Attacks detected: 0 (0.0%)
Benign flows: 413 (100.0%)

✓ All flows classified as BENIGN — your network looks clean!

Average inference time: 1.729ms
Max inference time: 6.661ms

Results saved to: live_analysis_results.csv
```

In [ ]: